

La visualisation 3D avec VPython

PAKA Fabrice

9 juin 2025

1 Introduction

Lorsqu'on crée des animations, on souhaite souvent y intégrer des mouvements en trois dimensions. Par exemple, le mouvement d'un ressort attaché à une masse, ou encore la trajectoire d'un projectile (comme une sphère) dans un champ de pesanteur.

Si, comme moi, vous aimez coder en Python et recherchez une bibliothèque simple à utiliser pour ce type de visualisation, je vous recommande **VPython** (Visual Python). C'est une librairie que j'ai récemment découverte et que j'utilise pour simuler et visualiser facilement et de façon interactive divers phénomènes physiques en 2D et en 3D.

Dans ce document, nous allons donc découvrir concrètement comment utiliser VPython à travers des exemples pratiques et illustratifs.

2 Installation de VPython

Tout d'abord il faut avoir déjà `python` d'installer (dans votre environnement virtuel ou non) et ensuite installer `vpython` via `pip` :

```
1 pip install vpython
```

Vous pouvez également écrire votre script sur trinket.io ou glowscript.org. Cela permettra à d'autres d'interagir avec la simulation de votre script.

3 Fonctionnement de VPython

VPython est une bibliothèque particulièrement efficace pour créer des objets en trois dimensions, des graphiques 2D ainsi que des animations interactives.

Son utilisation repose principalement sur la définition des objets à afficher, ainsi que sur la mise à jour de leurs propriétés dynamiques. **VPython** se charge automatiquement de la gestion de l'affichage : création des formes, rendu des ombres, animation en temps réel, etc.

3.1 Exemple de création d'objet en 3D

Voici un exemple minimal de création de sphère avec VPython :

```

1  #!/usr/bin/env python
2  from vpython import *
3
4  # Création de la scène
5  scene.background = color.white
6
7  # Création d'une sphère
8  ball = sphere(pos=vector(0,0,0), radius=0.5, color=color.red)
9

```

Ce script peut être exécuté directement dans un environnement Python (local). La fenêtre VPython s'ouvrira dans votre navigateur web (WebGL). L'image ci-dessous montre un aperçu de la scène 3D générée par le script précédent.

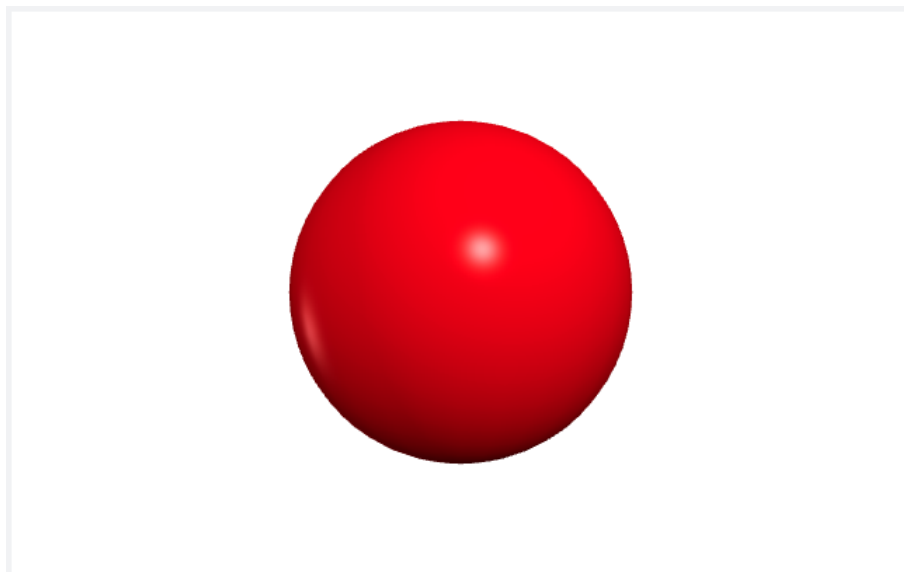


FIGURE 1 – Visualisation 3D d'une sphère avec VPython.

Notons que `vpython` dispose de plusieurs objets avec des attributs (couleurs, position, rayon pour la sphère) qui leur sont caractéristiques. Pour plus d'informations voir glowsript.org/3D-objects

4 Exemple de code pour l'animation en 3D

Réaliser une animation avec VPython est relativement simple. Le processus se déroule en deux étapes principales :

- **Création de la scène** : on commence par définir les objets 3D qui composeront l'environnement de l'animation.
- **Animation** : on met en place une boucle qui met à jour en temps réel les propriétés des objets (position, rotation, etc.) pour générer le mouvement.

Dans notre exemple ci-dessous nous allons illustrer le mouvement d'un objet en chute libre dans un champ de pesanteur $g = 9.81m/s$ lâché perpendiculairement au sol à une hauteur de 0.1 mètre au dessus du sol

```

1  #!/usr/bin/env python
2  from vpython import *
3
4  # Création de la scène
5  scene.background= color.blue
6  ball = sphere(pos=vector(0,.1,0), radius=0.05, color=color.yellow)
7  ground = box(pos=vector(0,0,0), size=vector(1.,0.02,0.25))
8
9  # definition des paramètres et initialisation des variables
10 g = vector(0,-9.8,0)
11 ball.m = 0.05
12 v0=3.5
13 ball.v = vector(0,v0,0)
14 dt = 0.01
15
16 # Boucle de simulation
17 while ball.pos.y > ball.radius+ground.size.y: # Vérifie si la balle est au
18     # dessus du sol
19
20     rate(100) # Limite la vitesse de la boucle pour qu'on puisse
21     # voir l'animation
22
23     # Mise à jour de la position et de la vitesse de la balle
24     ball.v = ball.v + a*dt
25     ball.pos = ball.pos + ball.v*dt
26

```

5 Création de graphiques 2D avec VPython

Aussi bien que `matplotlib`, `VPython` nous permet de faire des graphes en 2D. Cependant leurs méthodes de tracées sont opposées : `VPython` commence d'abord à créer un objet de graphique, puis à y ajouter les points un par un. En revanche, `matplotlib`, bien qu'il commence par créer un objet de graphique, crée ensuite un vecteur de points et trace l'ensemble d'un seul coup.

Cela etant voici un code de creation d'un graphique avec `VPython`

```

1  #!/usr/bin/env python
2  from vpython import *
3  # Creation de l'objet graphique ( axes et titres de notre graphique)
4  objet_graphique = graph(xtitle = 'time(s)', ytitle= 'x', title=f'Évolution de la position en
5  # creation de l'objet courbe
6  courbe =gcurve(color = color.red)
7  t = 0.
8  dt=0.01
9  v= 1.0
10 x = 0
11 a = 2.
12
13 while t<1:
14     v = v + a*dt
15     x = x +v*dt
16     # Construction et affichage du graphique
17     courbe.plot(x, t)
18     t = t+dt

```